

Guia Postman - Consulta de CVs en Google Cloud Storage

1. Alcance

Esta guia explica como usar Postman para consultar archivos CV almacenados en Google Cloud Storage.

Incluye:

- * Preparar un JWT firmado para OAuth 2.0.
- * Obtener un access_token.
- * Listar archivos del bucket.
- * Consultar metadata de un archivo.
- * Descargar o leer un archivo.

No incluye:

- * Upload de archivos.
- * Edicion de objetos.
- * Borrado de objetos.
- * Consultas a WordPress.
- * SQL.
- * Endpoints internos del sitio.

2. Flujo En Postman

El flujo recomendado en Postman es:

1. Preparar o recibir un JWT firmado con RS256.
2. Intercambiar el JWT por un access_token en Google OAuth.
3. Usar el access_token como Bearer token para llamar Google Cloud Storage.
4. Listar objetos del bucket, si se necesita ubicar archivos.
5. Consultar metadata de un objeto especifico.
6. Descargar el archivo usando el endpoint de descarga con alt=media.

3. Variables De Entorno En Postman

Crear un Environment en Postman con estas variables.

Variable	Descripcion	Ejemplo ficticio	Sensible
client_email	Email de la service account.	service-account@project-id.iam.gserviceaccount.com	Si
private_key	Private key de la service account.	-----BEGIN PRIVATE KEY-----...	Si, critica
jwt	JWT firmado con RS256.	eyJhbGciOiJSUzI1NiIs...	Si
access_token	Token OAuth temporal devuelto por Google.	ya29...	Si
bucket_name	Nombre del bucket de Google Cloud Storage.	careers-cvs-bucket	No, pero no publicar
object_name	Nombre/ruta del archivo dentro del bucket.	1710000000-uuid-cv.pdf	Puede ser sensible
object_name_encoded	object_name codificado para URL.	folder%2Fcv.pdf	Puede ser sensible

prefix	Prefijo opcional para filtrar listado.	171000	No
--------	--	--------	----

En las URLs de esta guía se usa la sintaxis de variables de Postman, por ejemplo:

```
{{bucket_name}}
{{access_token}}
{{object_name_encoded}}
```

4. Request 1 - Preparar JWT Firmado

Google OAuth requiere un JWT firmado con RS256 usando la private key de la service account.

Postman no debe recibir la private key completa salvo que sea estrictamente necesario. Para pruebas con cliente, la opción más segura es que el equipo técnico entregue uno de estos valores temporales:

- * Un access_token temporal listo para usar.
- * O un jwt firmado temporal.

Si el cliente necesita generar el JWT por su cuenta, debe usar una librería o script externo que firme RS256 con la private key de la service account. Este paso no es un request HTTP hacia Google Cloud Storage; es una preparación previa para poder ejecutar el Request 2.

Header JWT

```
{
  "alg": "RS256",
  "typ": "JWT"
}
```

Payload JWT

```
{
  "iss": "{{client_email}}",
  "scope": "https://www.googleapis.com/auth/devstorage.read_write",
  "aud": "https://oauth2.googleapis.com/token",
  "iat": 1710000000,
  "exp": 1710003600
}
```

Consideraciones

- * iss debe ser el email de la service account.
- * scope debe permitir lectura de Cloud Storage. En esta guía se usa https://www.googleapis.com/auth/devstorage.read_write.
- * aud debe ser <https://oauth2.googleapis.com/token>.
- * iat es la fecha/hora actual en Unix timestamp.
- * exp debe ser una fecha/hora futura. Recomendado: no más de 1 hora después de iat.
- * La firma debe usar RS256.
- * El JWT final se guarda en la variable de Postman jwt.

5. Request 2 - Obtener Access Token

Este request intercambia el JWT firmado por un access_token de Google OAuth.

Method

POST

URL

https://oauth2.googleapis.com/token

Headers

Content-Type: application/x-www-form-urlencoded

Body

En Postman:

1. Ir a Body.
2. Seleccionar x-www-form-urlencoded.
3. Agregar los campos:

Key	Value
grant_type	urn:ietf:params:oauth:grant-type:jwt-bearer
assertion	{{jwt}}

Respuesta Esperada

```
{
  "access_token": "{{access_token}}",
  "expires_in": 3599,
  "token_type": "Bearer"
}
```

Tests

Agregar este script en la pestaña Tests del request para guardar automaticamente el token:

```
const data = pm.response.json();

pm.test("Google OAuth devuelve access_token", function () {
  pm.expect(data.access_token).to.be.a("string").and.not.empty;
});

if (data.access_token) {
  pm.environment.set("access_token", data.access_token);
}
```

6. Request 3 - Listar Archivos Del Bucket

Este request lista objetos almacenados en el bucket.

Method

GET

URL

https://storage.googleapis.com/storage/v1/b/{{bucket_name}}/o

Headers

Authorization: Bearer {{access_token}}

Params

Opcionales:

Param	Value	Descripcion
prefix	{{prefix}}	Lista solo objetos cuyo nombre inicia con ese prefijo.
maxResults	100	Limita el numero de objetos devueltos.

Como Usar prefix

Si los objetos tienen nombres como:

```
2026/cv-001.pdf
2026/cv-002.pdf
2025/cv-999.pdf
```

Usar:

```
prefix=2026/
```

Para endpoints de listado, el prefix puede enviarse como texto normal en Params. Postman se encarga de codificarlo al enviar el request.

Respuesta Esperada

```
{
  "kind": "storage#objects",
  "items": [
    {
      "kind": "storage#object",
      "id": "{{bucket_name}}/{{object_name}}/{{generation}}",
      "name": "{{object_name}}",
      "bucket": "{{bucket_name}}",
      "generation": "{{generation}}",
      "contentType": "application/pdf",
      "size": "12345",
      "mediaLink": "https://storage.googleapis.com/download/storage/v1/b/{{bucket_name}}/o/{{object_name_encoded}}?generation={{generation}}&alt=media"
    }
  ]
}
```

El campo importante para identificar cada archivo es:

```
items[].name
```

Ese valor es el object_name.

Tests Opcional

Guardar el primer object name encontrado:

```
const data = pm.response.json();

pm.test("La respuesta contiene items", function () {
  pm.expect(data.items).to.be.an("array");
});

if (data.items && data.items.length > 0) {
  pm.environment.set("object_name", data.items[0].name);
  pm.environment.set("object_name_encoded", encodeURIComponent(data.items[0].name));
}
```

7. Request 4 - Consultar Metadata De Un Archivo

Este request consulta los datos tecnicos de un archivo especifico sin descargar el contenido.

Method

GET

URL

`https://storage.googleapis.com/storage/v1/b/{{bucket_name}}/o/{{object_name_encoded}}`

Headers

Authorization: Bearer {{access_token}}

Como Preparar object_name_encoded

El object name debe codificarse para URL.

Ejemplo:

```
object_name      = carpeta/cv-postulante.pdf
object_name_encoded = carpeta%2Fcv-postulante.pdf
```

La diagonal / debe convertirse en %2F cuando forma parte del nombre del objeto.

En Postman se puede generar desde Tests o Pre-request Script:

```
const objectName = pm.environment.get("object_name");

if (objectName) {
  pm.environment.set("object_name_encoded", encodeURIComponent(objectName));
}
```

Respuesta Esperada

```
{
  "kind": "storage#object",
  "id": "{{bucket_name}}/{{object_name}}/{{generation}}",
  "selfLink": "https://www.googleapis.com/storage/v1/b/{{bucket_name}}/o/{{object_name_encoded}}",
  "mediaLink": "https://storage.googleapis.com/download/storage/v1/b/{{bucket_name}}/o/{{object_name_encoded}}?generation={{generation}}&alt=media",
  "name": "{{object_name}}",
  "bucket": "{{bucket_name}}",
  "generation": "{{generation}}",
  "contentType": "application/pdf",
  "size": "12345"
}
```

Campos utiles:

- * name: nombre del objeto.
- * bucket: bucket donde esta almacenado.
- * contentType: tipo MIME.
- * size: tamaño en bytes.
- * mediaLink: URL de descarga generada por GCS.
- * generation: version/generacion del objeto.

8. Request 5 - Descargar O Leer Archivo

Este request descarga el contenido del archivo.

Method

GET

URL

`https://storage.googleapis.com/download/storage/v1/b/{{bucket_name}}/o/{{object_name_encoded}}`

Params

Param	Value
alt	media

Headers

Authorization: Bearer {{access_token}}

Como Probarlo En Postman

1. Ejecutar Request 2 para obtener access_token.
2. Definir bucket_name.
3. Definir object_name.
4. Generar object_name_encoded.
5. Ejecutar Request 5.
6. Usar Send para previsualizar si Postman soporta el tipo de archivo.
7. Usar Send and Download para guardar el archivo localmente.

Variante Con Token En Query Param

Tambien es posible enviar el token como query param:

`https://storage.googleapis.com/download/storage/v1/b/{{bucket_name}}/o/{{object_name_encoded}}?alt=media&access_token={{access_token}}`

Para pruebas en Postman se recomienda usar el header Authorization en lugar de poner el token en la URL. Esto reduce el riesgo de exponer tokens en capturas, historiales, logs o links compartidos.

9. Orden Recomendado De Ejecucion

Ejecutar los requests en este orden:

1. Request 1 - Preparar JWT firmado, o recibir un JWT/access token temporal.
2. Request 2 - Obtener access_token.
3. Request 3 - Listar archivos del bucket, si no se conoce el object_name.
4. Request 4 - Consultar metadata del archivo.
5. Request 5 - Descargar o leer el archivo.

Si el cliente ya tiene un access_token temporal, puede empezar directamente desde Request 3, Request 4 o Request 5.

10. Manejo De Errores Comunes

401 Unauthorized

Posibles causas:

- * access_token expirado.
- * Token no guardado correctamente en Postman.
- * Header Authorization ausente o mal formado.

Validar:

```
Authorization: Bearer {{access_token}}
```

403 Forbidden

Posibles causas:

- * La service account no tiene permisos suficientes sobre el bucket.
- * El scope del token no permite la operacion.
- * El bucket pertenece a otro proyecto o politica de acceso.

404 Not Found

Posibles causas:

- * bucket_name incorrecto.
- * object_name_encoded incorrecto.
- * El objeto no existe.
- * El object name contiene / y no se codifico como %2F.

400 Bad Request

Posibles causas:

- * JWT mal formado.
- * grant_type incorrecto.
- * assertion vacio.
- * iat o exp fuera de rango.

11. Recomendaciones De Seguridad

- * No compartir private_key por canales inseguros.
- * Preferir un access_token temporal para pruebas.
- * No dejar tokens, JWTs ni private keys en capturas de pantalla.
- * No guardar secretos reales en colecciones compartidas de Postman.
- * Usar ambientes separados para pruebas y produccion.
- * Revocar o rotar credenciales si se comparten por error.
- * Evitar URLs con access_token en query param cuando se compartan evidencias.